EE 446 TinyML Lab 6: Anomaly Detection

Luke Valerio

This report covers Part 1, fan anomaly detection built in Edge Impulse, and Part 2, a quantization-aware autoencoder for human-activity anomaly detection that runs on the Arduino Nano 33 BLE Sense.

Section 1: Edge Impulse Results (Part 1)

Public Edge Impulse project link

https://studio.edgeimpulse.com/public/1076612/latest

Answers to the required questions

1. The raw inputs are the three accelerometer axes accX, accY, and accZ, taken as a time series sampled at 100 Hz with a 2000 ms window and a 220 ms stride (zero-padded).
2. The classifier is trained on spectral features from the Spectral Analysis DSP block. There are 33 input features in total; five of them are accX RMS, accX Peak 1 Height, accX Spectral Power 0.1 to 0.5, accY RMS, and accZ Spectral Power 2.0 to 5.0, alongside the remaining per-axis peak, frequency, and spectral-power values.
3. The classifier outputs two classes, nominal and off. The impulse as a whole reports three outputs (nominal, off, and an anomaly score) because the K-means block adds the anomaly score on top of the classifier.
4. The anomaly detector is a K-means model trained on a selected subset of the same spectral features; here it uses three, accX Peak 1 Height, accY Spectral Power 2.0 to 5.0, and accZ Spectral Power 2.0 to 5.0, and it outputs a single anomaly score.

Deployment evidence

```
  nominal: 0.656250
  off: 0.343750
Anomaly prediction: 56.150742
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 581 us, anomaly 571 us, postprocessing 388 us
#Classification predictions:
  nominal: 0.000000
  off: 0.996094
Anomaly prediction: 10.294332
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 523 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.597656
  off: 0.402344
Anomaly prediction: 757.701416
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 522 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.023437
  off: 0.976562
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 551 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.921875
  off: 0.078125
Anomaly prediction: 50.320477
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 529 us, postprocessing 78 us
#Classification predictions:
  nominal: 0.824219
  off: 0.175781
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 537 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.140625
  off: 0.859375
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 45 ms, inference 562 us, anomaly 549 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.562500
  off: 0.437500
Anomaly prediction: 15.089006
Starting inferencing in 2 seconds...
```

Live inference on the Arduino Nano 33 BLE Sense using the Edge Impulse run-impulse command. The board produces both classifier states, a strong off reading (for example 0.996) and a strong nominal reading (for example 0.922), and it reports the per-window timing (DSP 44 ms, classifier inference 6 ms, anomaly about 0.55 ms). The K-means anomaly score climbs steeply, up to 1000, once the motion falls outside the training distribution.

Short interpretation

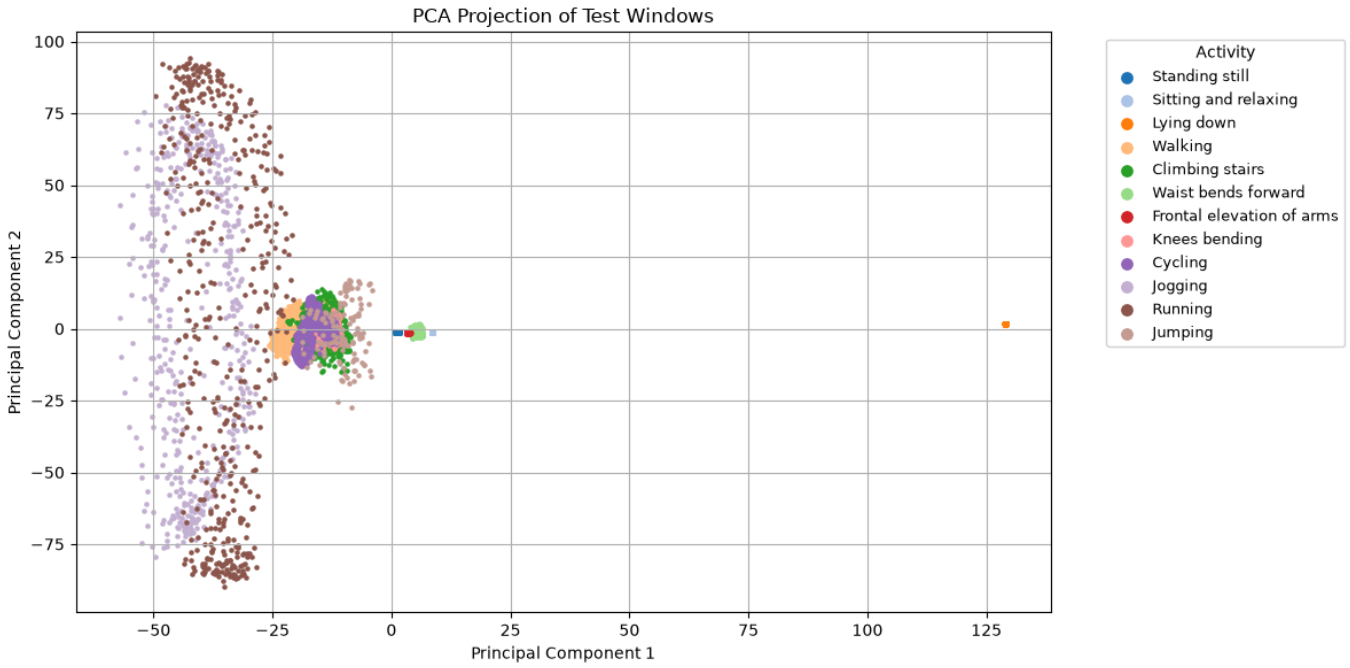
The reference classifier reaches 100 percent validation accuracy and an ROC-AUC of 1.00. That looks perfect, but I would not trust it unconditionally. The dataset is small and the windows overlap, so the validation split can leak information and make the accuracy look better than it really is, and a different fan, mounting, or speed would probably lower it. The classifier is dependable mainly when the input resembles the training data, which is exactly what the K-means anomaly score measures. In practice I would trust the nominal or off label when the anomaly score is low and the operating conditions match those seen during training, and I would treat the label as unreliable when the anomaly score is high, or when the deployment hardware and environment differ from the setup the model was trained on.

Section 2: Autoencoder Model Results (Part 2)

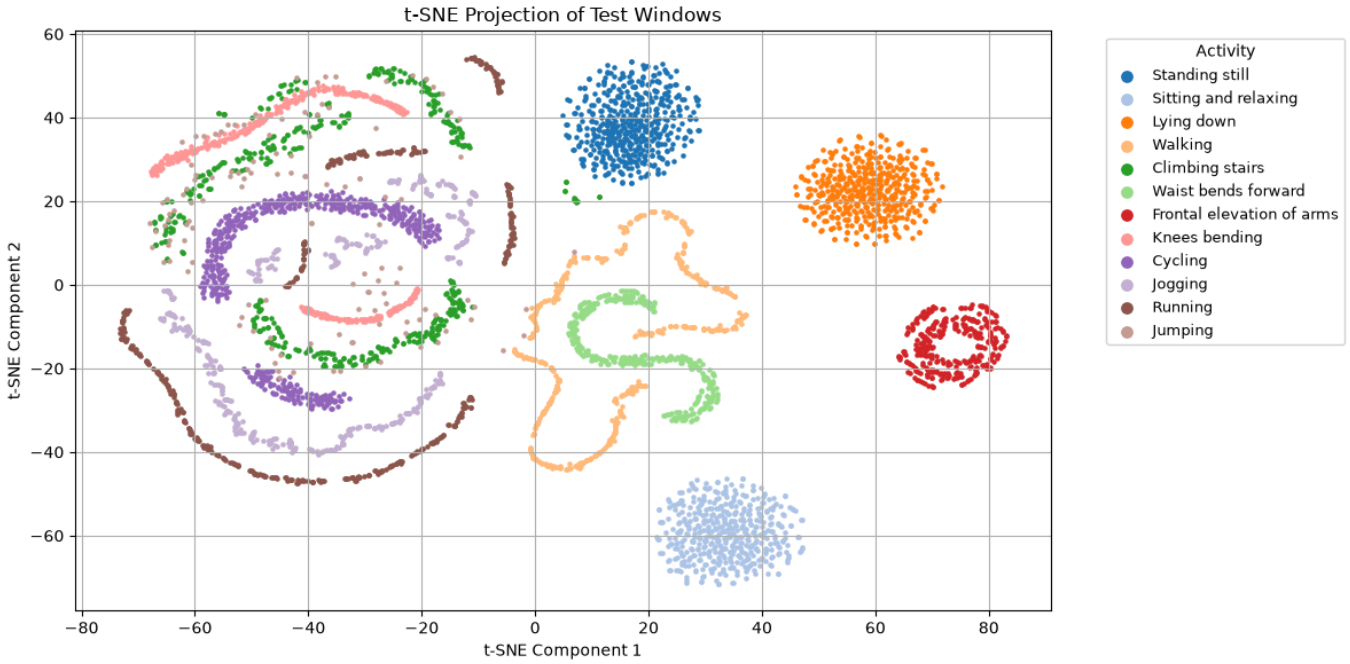
Data preparation

Input channels	Left-ankle accelerometer X, Y, Z (mHealth columns 6 to 8); activity label in column 24.
Windowing	100 samples by 3 axes, flattened into 300-dimensional vectors (stride 1), with a 70/30 temporal split inside each class.
Normal group	standing, sitting, lying, waist bends, frontal arm elevation, knees bending (labels 0, 1, 2, 5, 6, 7).
Train windows	21,350 (10,478 normal, 10,872 anomaly).
Test windows	8,479 (4,155 normal, 4,324 anomaly).

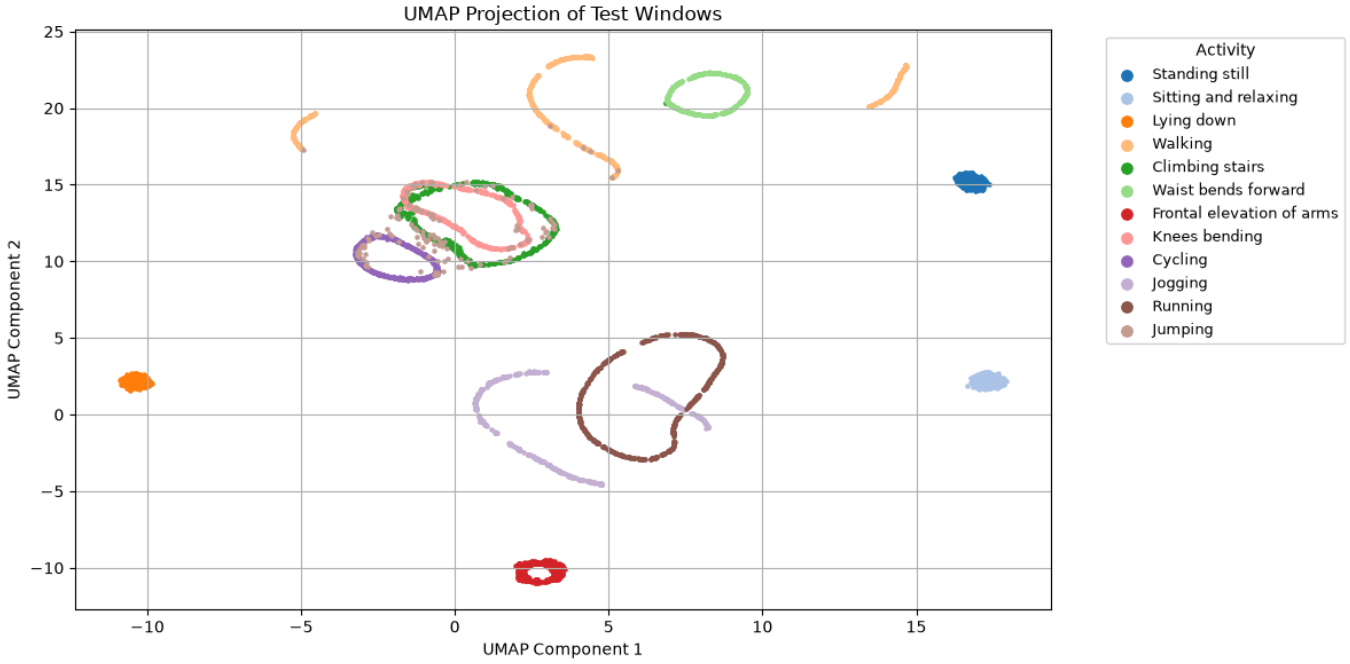
Dimensionality-reduction visualizations



PCA projection of the test windows, colored by activity.

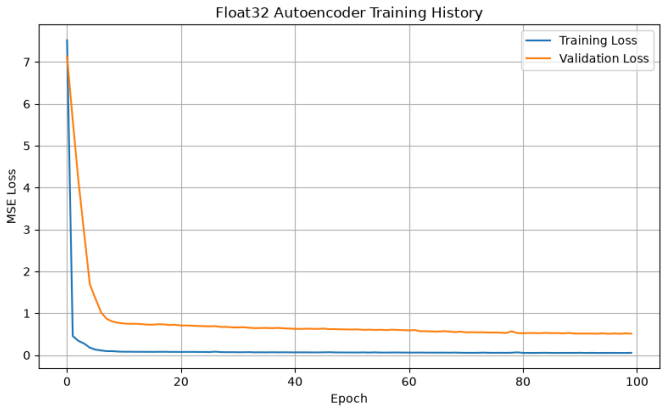


t-SNE projection of the test windows.

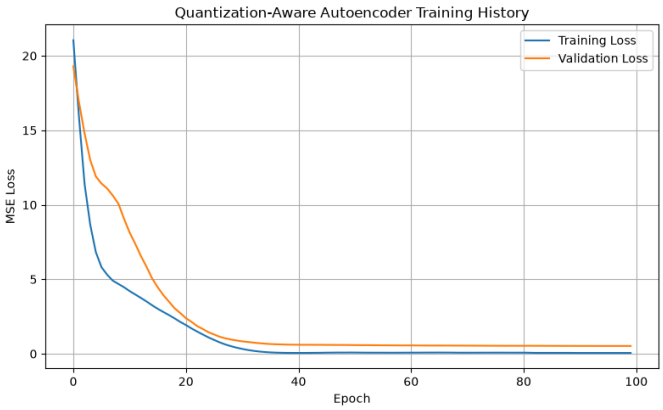


UMAP projection of the test windows.

Training and validation loss curves

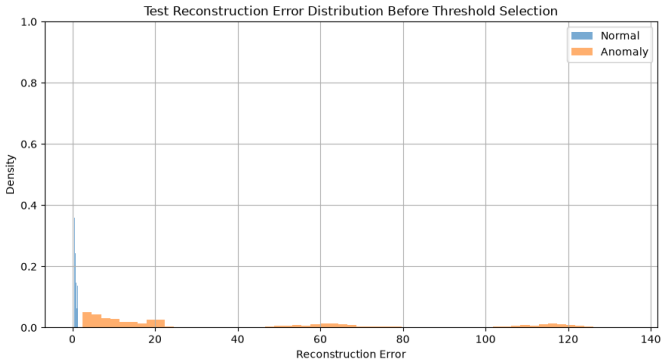


Float32 autoencoder training.

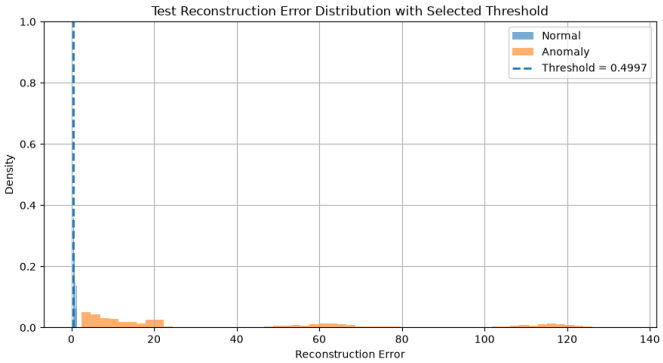


Quantization-aware training.

Reconstruction error distribution and selected threshold



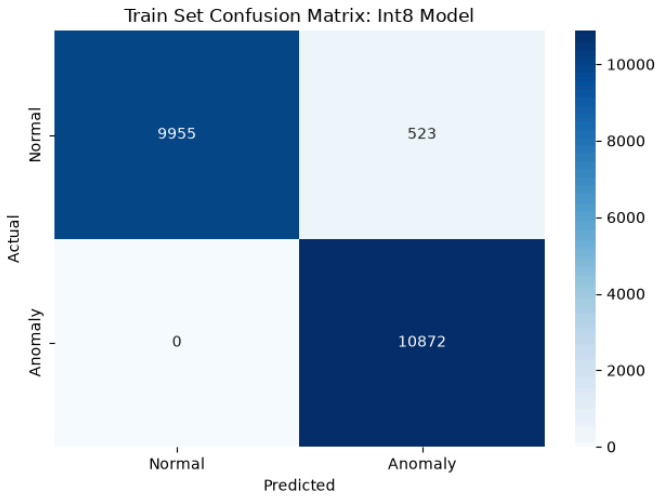
Before choosing a threshold.



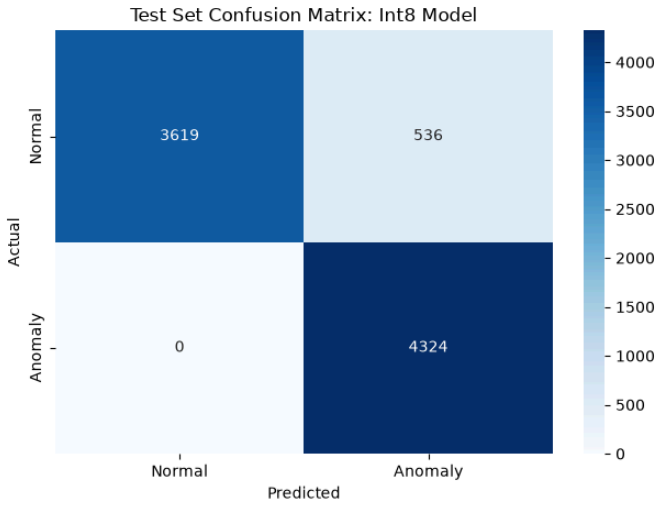
With the selected threshold.

Selected threshold	0.50 (mean squared error), the 95th percentile of reconstruction error on normal training windows.
Train normal error	mean 0.105, max 1.105.
Test normal error	mean 0.145.
Test anomaly error	mean 43.0, min 2.53, roughly 300 times higher than the normal windows.

Confusion matrices and evaluation metrics (int8 model)



Training set.

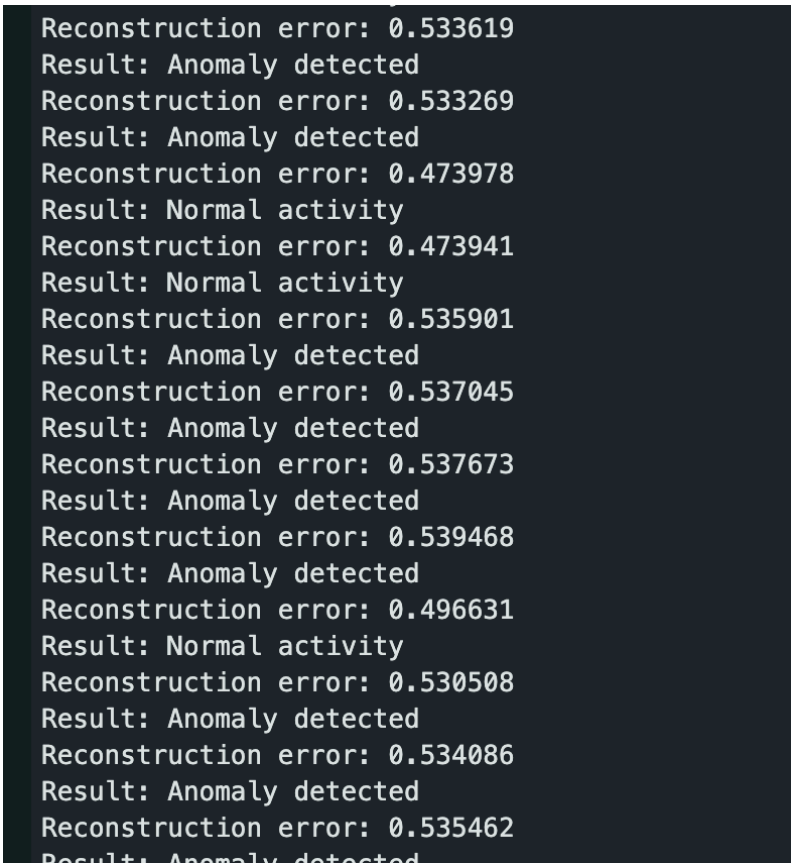


Test set.

```
Test set evaluation (int8 model)
              precision    recall  f1-score   support
  Normal (0)       1.00      0.87      0.93      4155
 Anomaly (1)       0.89      1.00      0.94      4324
    accuracy                           0.94      8479
   macro avg       0.94      0.94      0.94      8479
weighted avg       0.94      0.94      0.94      8479
```

Input quantization	scale 0.08136, zero-point 18.
Output quantization	scale 0.08585, zero-point -8.
Model size	82.87 KB as float32 TFLite, 24.46 KB as full int8 (g_model_len is 25,048 bytes).

Arduino serial monitor output



The autoencoder running on the Arduino Nano 33 BLE Sense, read over the serial monitor at 115200 baud. Held near the ankle-normal orientation, the reconstruction error sits close to the 0.50 threshold: windows below 0.50 (for example 0.474 and 0.497) are labeled Normal activity, and windows just above it (for example 0.531 to 0.539) are labeled Anomaly detected. This shows the decision boundary working live on the device.

On the board, the reconstruction error is driven mostly by sensor orientation. Lying flat on a table gives about 90, a hand-held vertical pose gives about 2.5 to 5, and only a carefully held ankle-like orientation drops below 0.50 into the normal range. This is a concrete illustration of why the deployed sensor has to match the mounting and units used during training, which I come back to in the discussion.

Section 3: Discussion Questions

What threshold did you choose for anomaly detection, and why?

I chose 0.50 mean squared error, set at the 95th percentile of the reconstruction error on normal training windows. Because the autoencoder only ever sees normal data during training, about 95 percent of normal windows fall below this value, which keeps the false-positive rate low. At the same time the gap to the anomaly errors is large, since their minimum is about 2.53 and their mean about 43, so every anomaly in the test set is still caught (anomaly recall of 1.00). The choice is fully unsupervised because it never looks at any anomaly labels.

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder learns to compress and rebuild the normal activity patterns, so it reconstructs them with very low error, around 0.1. Higher-motion, anomalous activities fall outside that learned pattern, so the decoder rebuilds them poorly and the error jumps by roughly 300 times. Because the error is a single number, I can separate normal from anomalous just by thresholding it, without ever training on a labeled anomaly class.

What information from the Python notebook must be preserved when deploying the model to Arduino?

Several things from the notebook have to carry over to the sketch. The first is the int8 model bytes in autoencoder_model.cc, held in the g_model array. The second is the input and output quantization parameters, the scale and zero-point, which the sketch reads back from the model at runtime. The third is the window geometry, 100 samples by 3 axes stored in row-major order (index i times 3 plus j), and the reconstruction-error threshold of 0.50. Finally, the definition of the input itself matters: a left-ankle three-axis accelerometer in meters per second squared, sampled at about 50 Hz.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The int8 quantization is calibrated to the training distribution, so if the board feeds data at a different scale, axis order, or window length, the same physical motion turns into different int8 values than the model expects. When that happens the reconstruction error, and therefore the 0.50 threshold, no longer separates the classes. Units are a concrete example: the BMI270 reports acceleration in g, while the mHealth data is in meters per second squared, so the sketch multiplies each axis by 9.80665 to match. The flatten order and the 100-sample window length have to match as well.

What are the main limitations of this method when deployed on a microcontroller?

There are several. The threshold is fixed, so sensor drift or a different wearer would shift the error distribution and throw off the decision. The int8 rounding adds noise to the reconstruction, so windows near the threshold are easy to misclassify. The model was trained on a single subject, and the split into normal and anomalous is really just an arbitrary grouping of activities rather than genuine fault detection, so it does not generalize far. It also relies on a single sensor at a fixed rate, the left-ankle accelerometer with 100-sample windows at about 50 Hz, and it is quite sensitive to orientation, as the deployment showed. Finally, there is no on-device learning and only limited memory, a 64 KB tensor arena and a 24.5 KB model in flash, and the detector really only flags novelty rather than anything semantically meaningful.